



S.G. Ganesh

C Puzzlers: Traps, Pitfalls and Corner Cases

Recently, I read an interesting book* on Java programming puzzles. Since a few of them would interest programmers knowledgeable in C-based languages, I am covering three puzzles from this book.

Assume the following: integer 4 bytes size and long 8 bytes size; the underlying machine uses two's complement representation for integers; the necessary header files are included; and the C compiler supports the C99 standard.

If you're a C, C++, Java, C# or even D programmer, you will enjoy these puzzles. Be warned: though they look harmless and appear to work fine, they however have bugs, so the results you see after executing these programs will surprise you!

- 1) This program is about implementing a simple digital clock. The `loop` variable 'millis' is for counting from 0 to the number of milliseconds in an hour. The variable 'mins' is to count minutes. The `printf` prints the number of minutes in an hour after executing the loop. So what is the output of this program? (Hint: There are 60 minutes in an hour).

```
int main() {
    int mins = 0;
    for(int millis = 0; millis < 60*60*1000; millis++)
        if(millis % 60*1000 == 0)
            mins++;
    printf("%d", mins);
}
```

- 2) What is the output of this program? (Hint: It is a simple program to check if you've learned addition at school!)

```
int main() {
    printf("%lx", 0x10000000L + 0xCAFEBAE);
}
```

- 3) You've written this program to search for articles on open source on the *LFY* website. Will it open Firefox or IE on your Linux machine? (Hint: IE is not available on Linux).

```
int main() {
    http://www.linuxforu.com/
    printf("Linux + open source");
}
```

Answers:

- 1) You would expect the program to print 60, since one hour has 60 minutes. However, it prints 60000. This is the problem: the expression is evaluated as $(\text{millis} \% 60) * 1000$ and not as $\text{millis} \% (60 * 1000)$. Remember that `%` has the same precedence as the `*` operator, so the `%` operator is evaluated before `*`.

Now, since $(\text{millis} \% 60)$ is 0 for $60 * 1000$ times in the loop, the expression `if(millis % 60*1000 == 0)` is true for 60000 times and hence the output.

- 2) You would expect this program to print 1CAFEBAE, because we are adding 0x10000000 to 0xCAFEBAE. However, you'd certainly be surprised with the answer: it prints only CAFEBAE! Why?

0xCAFEBAE is a negative integer constant! When we write negative constants in decimal form, like "-10", it is very clear that the constant is negative because we can see the preceding "-" sign. However, for hex and octal numbers, the number is negative if the highest order bit is negative. In this case, 0xCAFEBAE is a negative number. Here is a quick way to confirm it. If you run the following statement: `"printf("%d", 0xCAFEBAE);"`, you'll get the value "-889275714" as the output. Now, what happens to the addition?

Note that 0x10000000L is a long number. When 0xCAFEBAE is promoted to long, it is prefixed with 1's (i.e. the sign is extended). So it becomes 0xFFFFFFFFCAFEBAE. When we add 0x10000000L with 0xFFFFFFFFCAFEBAE, we get 0xCAFEBAE (remember, adding 1 to F is 0, with a carry 1). Hence the output!

- 3) This program prints "Linux + open source" in the command line. What happens to the URL `http://www.linuxforu.com/?` Well, if you check your compiler warnings, you might get something like: unreferenced label 'http' in function main. 'http' became a label (labels are used as the target for `goto` statements) and `//` became the starting of a single line comment! (C99 supports C++ style single line comments). 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.

(*) 'Java puzzlers: Traps, Pitfalls, and Corner Cases', Joshua Bloch, Neal Gafter, Addison-Wesley, 2005.